L4 DDoS Detection: Adaptive Thresholds and Attack Lifecycle Plan

Oct 20, 2026 · @Anton Volkodav

Summary

Revised after reviewing Bit-Warrior-X/ddos-firewall/ebpf_firewall (`kernel/ebpf_firewall_kernel.c`, `common.h`, `userspace/src/ebpf_firewall_core.c`, `ebpf_firewall_conntrack.c`, `firewall.config`). The current design detects attacks with global per-protocol pps counters, then blocks individual source IPs for 1 hour using per-source burst and fixed-window counters.

That model has three structural problems: STOMP and connection floods pass through the established-connection fast path untouched; most blocks are based on spoofable packets, so attackers can blacklist legitimate users; and, since 0.4.0, PSH+ACK protection starts on host-wide PSH+ACK volume, which legitimate CDN traffic can reach. The redesign keeps your XDP + C daemon structure and changes what is counted, what is trusted, and how mode is decided:

- **Fix critical bugs first** (section 2): the established-map reply-tuple bypass, conntrack loose pickup, the IPv6/VLAN bypass, and the 0.4.0 PSH+ACK gate and exclusive STOMP/HANDSHAKE modes (the SYN worker race is fixed in 0.4.0).
- **Validate, don't just count.** Every packet class gets a validity test (socket lookup, SYN cookie, outbound-SYN match, port/peer allowlist). Invalid traffic is the primary trigger and the first thing dropped.
- **Rate-limit established traffic.** A per-flow and per-source layer catches STOMP, HANDSHAKE and SOCKET floods that use real handshakes.
- **Block sources only on proof.** A source IP is blacklisted only after it completed a handshake or passed a cookie; spoofable floods get dropped, not blacklisted.
- **Adaptive thresholds per destination**, a capacity ceiling derived per node from NIC speed and CPU, and one hysteresis state machine in the C daemon.

The XDP program stays simple: validate, count, and enforce a mode. All statistics and decisions move into one control loop in the existing C userspace, so tuning never requires a BPF reload.

Scope: L3/L4 only, for single-vector and combined (multi-vector) attacks up to link capacity. L7 (HTTP, and QUIC requests after a completed handshake) is handled in OpenResty. Floods above link capacity are handed upstream (see Upstream escalation).

Current implementation review

Re-reviewed against version 0.4.0 (commit `3a903b2`, 2026-10-20). 0.4.0 fixes the SYN worker race, moves traffic stats to monotonic deltas, stops fast-pathing PSH+ACK during STOMP/HANDSHAKE protection, and splits PSH+ACK into STOMP and HANDSHAKE by payload-length profile. Two critical bypasses (C2, C5) remain open, and the new PSH+ACK logic adds two high-severity risks (N1, N2). File references are to `ebpf_firewall/`.

#	Severity	Status in 0.4.0	Finding	Where	Fix
C1	Critical	Fixed	`global_attack_syn_check_worker` (1 ms read-and-zero of the SYN counter) is removed; one worker decides SYN mode.	`userspace/src/ebpf_firewall_core.c`	Done. Writes of the whole `global_fw_config` to `.bss` remain; write only mode fields.
C2	Critical	Open	`tcp_established_map` is keyed by `(src_ip, src_port)` and conntrack still inserts the reply tuple (`key_repl`). For inbound connections that key is `(server IP, 443)`; for outbound ones `(origin IP, origin port)`. Any non-SYN packet spoofed from those tuples skips every check, to any destination port.	`ebpf_firewall_conntrack.c`, `is_tcp_established()`	Key by full inbound 4-tuple; insert only the inbound direction. Prefer `bpf_skc_lookup_tcp()`.
C3	Critical	Partly fixed	PSH+ACK is no longer fast-pathed while `stomp_attack` or `handshake_attack` is on, and the gate uses `aggregate.psh_ack`, counted before the fast path. Still open: data segments without PSH (plain ACK with payload, FIN+ACK or URG+ACK with payload) are always fast-pathed on established tuples, so clearing the PSH bit evades STOMP and HANDSHAKE entirely.	`xdp_main` fast path	Classify on payload length > 0, not on the PSH bit (`DATA` class); apply layer 4 to all data segments.
C4	Critical	Partly fixed	`conntrack_saw_syn()` skips flows whose conntrack flags lack `0x02`. But `ATTR_TCP_FLAGS_ORIG/REPL` are conntrack's internal flags, where `0x02` is `IP_CT_TCP_FLAG_SACK_PERM`, not the TCP SYN bit. It works only as a proxy: loose pickups never see SACK options, but real clients that do not offer SACK lose the fast path. `nf_conntrack_tcp_loose` is still 1.	`ebpf_firewall_conntrack.c`, `99-ddos-firewall.conf`	Set `nf_conntrack_tcp_loose = 0`; or test `IP_CT_TCP_FLAG_BE_LIBERAL` (`0x08`), which conntrack sets on loose pickup.
C5	Critical	Open	IPv6 and VLAN-tagged frames still pass unfiltered; 0.4.0 now counts them as `other` bytes, so they are visible but not inspected.	`xdp_main` and every handler	Parse 802.1Q/QinQ, then IPv4 and IPv6 (with extension headers).

#	Severity	Status in 0.4.0	Finding	Where	Fix
N1	High	New	The PSH+ACK gate is host-wide: `aggregate.psh_ack >= min(stomp_threshold, handshake_threshold)` = 10,000 pps. That counter includes all legitimate client data (HTTP requests, TLS records, uploads, WebSockets), which a CDN node exceeds at normal peak. Protection then sends every PSH+ACK, including established flows, through a 200 packets per 10 s per-source window (20 pps) and blocks offenders for 1 h.	`global_attack_check_worker`, `firewall.config`	Gate on abnormal PSH+ACK only: out-of-state (socket miss) plus per-flow and per-source rates above the destination baseline (layer 4). Per-source limits from the baseline, not a fixed 20 pps.
N2	High	New	STOMP and HANDSHAKE are mutually exclusive and sticky. Userspace picks one by comparing `total_handshake_cnt` with `total_stomp_cnt`; once one is active, `xdp_parse_psh_ack` routes all PSH+ACK to that handler, so the other counter drops to near zero and a switch or a combined STOMP + HANDSHAKE attack is never seen. `total_handshake_cnt` also includes SYN+ACK packets, which skews the choice.	`xdp_parse_psh_ack`, `global_attack_check_worker`	Count both labels independently before any routing; allow both modes at once; move SYN+ACK out of the HANDSHAKE counter (H7).
N3	Medium	New	The payload-length label is decided once per source after 32 packets and never re-evaluated (`klass` is permanent until LRU eviction). Length is attacker-controlled: randomizing it turns STOMP into HANDSHAKE at will, and a source behind CGNAT mixes many users into one profile.	`psh_len_observe()`	Use the length profile as a label for reporting only; enforcement must not depend on it. Re-evaluate the profile per window.
N4	Medium	New	Payload length is computed from IP `tot_len` without checking it against the frame length.	`xdp_parse_psh_ack`	Stage 0: drop when `tot_len` exceeds the frame or headers.
H1	High	Open	Sources are blacklisted for 1 h on spoofable evidence (ACK, RST, UDP, ICMP, GRE, SYN). Attackers can spoof customers, CGNAT gateways, resolvers or origins to get them blocked; random spoofs fill the 524K `blocked_ips` LRU.	`block_ip_with_event()` callers	Blacklist only proven sources (completed handshake, valid cookie, established flow).
H2	High	Partly fixed	STOMP and HANDSHAKE drops are now counted in `global_psh_ack_filtered`, but per-class `pkt_cnt` in `DEFINE_FLOOD_HANDLER` is still incremented only on pass, and `blocked_ips` drops for other classes are not counted. The STOMP/HANDSHAKE choice (N2) uses these post-verdict counters.	`DEFINE_FLOOD_HANDLER`, `xdp_main`	Count once at `xdp_main` entry, before any verdict.
H3	High	Partly fixed	Traffic stats now use monotonic deltas. Attack counters (`global_*_pkt_counter`, `global_income_pkt_counter`, `global_psh_ack_filtered`) are still read and then zeroed, losing increments in between.	`global_attack_check_worker`	Same delta approach as the traffic-stats worker.
H4	High	Open	The SYN challenge (drop first SYN, pass the retransmit) is bypassed by sending each spoofed SYN twice and costs every legitimate new client at least 1 s.	`xdp_parse_syn`	XDP SYN cookies (kernel 6.0+), or kernel syncookies with XDP shedding.

#	Severity	Status in 0.4.0	Finding	Where	Fix
H5	High	Open	SYN+FIN, SYN+RST, SYN+URG still match `syn && !ack` and go to the SYN path.	`xdp_main` dispatch	Flag lookup table before dispatch.
H6	High	Corrected	The earlier review said per-source windows run outside attack mode; that was true only for SYN, and 0.4.0 removes it. Generic handlers enforce only in attack mode. Remaining risk is the size of the limits in attack mode: UDP 50 pps, PSH+ACK 20 pps, ICMP and RST 10 pps per source, each with a 1 h block.	`firewall.config`	Per-source limits only on validated traffic, sized from the destination baseline.
H7	High	Open	Inbound SYN+ACK is still dispatched to key 8 together with variable-length PSH+ACK, and shares its counter. SYN-ACKs for the server's own outbound connections (CDN to origin) share limits with reflected SYN-ACK floods.	`xdp_main` dispatch, `xdp_parse_handshake`	New SYN-ACK handler (key 10) that accepts only SYN-ACKs matching an outbound SYN recorded at TC egress.
M1	Medium	Open	Every counter is one global `PERCPU_ARRAY` key: one attacked VIP switches the whole host into attack mode.	All `global_*` counter maps	Key counters by `{dst, class}`.
M2	Medium	Open	`__sync_fetch_and_add` on per-CPU values, now also in `count_psh_ack_filtered()`.	All handlers	Plain increment on per-CPU memory.
M3	Medium	Open	Only non-first fragments are dropped; first fragments and fragment rates are unchecked.	`tcp_seg_check` block	Drop TCP fragments always; rate-limit fragments per destination.
M4	Medium	Open	`established_map_remove_all_for_ip` restarts a full walk after every delete: O(n^2) on a 1M map.	`ebpf_firewall_conntrack.c`	Goes away with 4-tuple keys and LRU expiry.
M5	Medium	Open	No sanity checks for `ihl < 5`, TCP `doff < 5`, IP total length vs frame, land, or bogon sources.	`xdp_main`	Stateless stage 0 before dispatch.
M6	Medium	Mostly fixed	`report_attack` now includes STOMP and HANDSHAKE, and byte counters see whitelisted traffic. Per-class packet counters still skip `allowed_ips`.	`ebpf_firewall_core.c`	Count packets before the whitelist too.
M7	Medium	Open	Burst-gap macros in `common.h` lack parentheses; runtime values come from `firewall.config`.	`common.h`	Parenthesize or remove.

Quick wins on 0.4.0: set `nf_conntrack_tcp_loose = 0` (C4), stop inserting the reply tuple (C2 short-term), switch attack counters to deltas (H3), and raise the PSH+ACK gate and per-source limits until layer 4 exists (N1). C3, C5, N2 and H1 need the redesign below.

N5 (High, found while tuning the config): the per-source burst detector never fires. `ebpf_firewall_config.c` sets `burst_gap_ns = 1 s / burst_count_per_sec`. Reaching N bursts needs N gaps longer than 1/N s, which is more than the 1 s window, and the window resets before the Nth burst is counted. This holds for every class, so the fixed window is the only per-source enforcement in 0.4.0. Fix: derive the gap independently (e.g. a fixed 50 ms) or drop burst detection in favor of the layer 4 per-source buckets.

Detection architecture: three signal layers

An attack is declared when any layer fires; each layer covers a gap the others leave.

Layer	Question it answers	Scope	Source of threshold	Catches
Stage 0. Stateless sanity	Is this packet ever legitimate?	Every packet	None; fixed rules	Malformed headers, invalid flags, land, bogons, disallowed protocols
1. Capacity ceiling	Will this volume hurt the machine?	Per host and per destination, pps and bps	Derived from NIC speed, driver, queues and CPU (Hardware section)	Any flood large enough to degrade the box, even if it looks legitimate
2. Adaptive baseline	Is this unusual for this destination?	Per (dst, class)	EWMA mean and variance, learned; bounded by class floor and cap	Floods well above normal that stay under capacity
3. Invalid traffic	Is this traffic invalid?	Per (dst, class)	Near-zero by definition	Floods that mimic normal volume; the primary trigger
4. Established-flow rate	Are real connections being abused?	Per flow, per source, per /24	Destination baseline for flow and source rates	STOMP, HANDSHAKE, SOCKET

Layer 1: capacity ceiling

Computed per node from NIC speed, driver, RX queues and CPU (see Hardware: NIC speed and derived parameters): 65% of the smaller of the 64-byte line rate and the cores' pipeline rate, plus 80% of the smaller of link and transit capacity in bps. A load test per hardware profile supplies the per-core rate; re-measure after kernel, driver or NIC changes.

Layer 1 is evaluated on totals, not only per class, because a combined attack can keep every class under its own threshold:

- **Per destination, all classes summed:** pps and bps against the destination's ceiling.
- **Per host, all destinations summed:** pps and bps against the NIC and CPU ceiling, and bps against 80% of link capacity.
- **NIC drops before XDP:** deltas of `rx_missed_errors`, `rx_no_buffer_count`, `rx_fifo_errors` (driver-specific names, read via `ethtool -S` or `/sys/class/net/*/statistics`). Any sustained increase means XDP is no longer seeing all traffic, so it triggers Incident on every destination with rising traffic.
- **XDP CPU budget:** softirq time per core from `/proc/stat`; above 80% for 3 s is treated like a ceiling breach.

Layer 2: adaptive baseline

Sample each counter every 1 s and update:

```
\mu \leftarrow \mu + \alpha (r - \mu)
```

```
\sigma^2 \leftarrow (1 - \alpha)\left(\sigma^2 + \alpha (r - \mu)^2\right)
```

```
T_{enter} = \max(\mu + k\sigma,\; m\mu,\; floor)
```

- Horizon: half-life of 30–60 minutes (alpha about 0.0002–0.0004 at 1 s samples).
- Seasonality: keep 168 hour-of-week buckets where traffic has daily or weekly cycles; compare against the matching bucket.
- Guard: cap baseline growth at 2x per hour so slow ramps cannot raise it. Add a CUSUM detector on the same series for slow-ramp attacks.
- Bootstrap: 7-day learning period; only layers 1 and 3 enforce until it completes.

Layer 3: semantic signals

Count invalid traffic separately from total traffic. Healthy servers produce very little of it, so the threshold can be a small absolute floor plus a ratio, and it holds across servers of any size. Per-protocol definitions follow.

Layer 4: established-flow rate (new)

Valid traffic can still be an attack. STOMP, HANDSHAKE and SOCKET floods complete real handshakes, so layers 2 and 3 see nothing invalid, and today's fast path passes them unchecked (C3). Track established TCP in an LRU map keyed by 4-tuple: pps, bytes,

payload packets and last-seen time per flow, plus an aggregate per source and per source /24.

- Thresholds come from the same EWMA machinery, keyed per destination service.
- Caps apply only in Attack, so normal heavy clients are never throttled.
- Because these sources proved reachability, this is the one layer where source blacklisting is safe.

Stage 0: stateless sanity (before any layer)

Drop packets that are never legitimate, with no state and no thresholds: malformed headers (`ihl < 5`, `doff < 5`, length mismatch), invalid TCP flag combinations, land packets, bogon or own-prefix sources, TCP fragments, and IP protocols not in the destination's allowlist. Count each drop reason; spikes in these counters are attack signals on their own.

Attack catalog and detection signals

Each attack gets a validity test that defines its invalid counter, which is the trigger to trust; volume counters back it up. The names follow your code: STOMP is a PSH+ACK flood, HANDSHAKE is a completed handshake followed by PSH+ACK data (connection churn), SYN-ACK is a new separate class for inbound SYN+ACK floods, and SOCKET is redefined below as connection exhaustion, with the current flag-anomaly handler moving to stage 0.

Required attacks

Attack	What it is	Current gap (0.4.0)	Detection signal	Mitigation	Blacklist source?
UDP	Random-port volumetric, or amplification reflected from 53, 123, 19, 161, 389, 1900, 3702, 11211	Global count only; 50 pps per-source limit in attack mode blocks QUIC (H6)	Packets to ports with no listener; packets from reflection source ports to non-client ports; per-dst pps vs baseline	Per-dst port allowlist map; drop reflection source ports unless the host is their client; QUIC on 443: check header form, Initial >= 1200 bytes; per-source bucket in Attack	No (spoofable)
GRE	GRE-IP / GRE-ETH floods (Mirai) carrying random inner packets	Rate-only, no peer check, no header check	Any GRE from a non-configured peer (normally 0)	Peer allowlist; drop all GRE when no tunnels are configured	No

Attack	What it is	Current gap (0.4.0)	Detection signal	Mitigation	Blacklist source?
ICMP	Echo floods, BlackNurse (type 3 code 3), oversized or fragmented ICMP	All types treated alike; 10 pps per source	Rate per type; echo-replies with no matching request; type 3 for flows that do not exist	Type allowlist; echo-request bucket per dst; drop ICMP fragments	No
TCP SYN	Spoofed SYNs exhausting the backlog	Retransmit challenge is bypassable and adds 1 s latency (H4)	SYN : completed-handshake ratio per dst; `ListenDrops` / `ListenOverflows` from `/proc/net/netstat`	XDP SYN cookies in Attack; kernel `tcp_syncookies` as fallback	No
TCP ACK	Out-of-state ACKs to exhaust stack and conntrack	Loose pickup only partly blocked (C4)	ACK whose 4-tuple has no socket: `bpf_skc_lookup_tcp()` miss, or listener hit with failed `bpf_tcp_check_syncookie()`	Drop invalid in Attack (mode 1)	No
STOMP (PSH+ACK)	Data flood with a constant payload length. Mirai-style: one real handshake, then a fixed-size PSH+ACK flood with valid sequence numbers on that connection. Also out-of-state PSH+ACK from raw sockets	Labeled STOMP when a source keeps one payload length over 32 packets. Gate is the host-wide PSH+ACK rate, which legitimate traffic can cross (N1); clearing the PSH bit evades it (C3); exclusive with HANDSHAKE (N2)	Out-of-state: socket lookup miss. In-connection: per-flow data pps/bps far above the destination baseline on a small number of flows. Constant payload length as a supporting label	Drop out-of-state; per-flow bucket; close abusive sockets (`bpf_sock_destroy`, kernel 6.5+, or `ss -K`)	Yes, for in-connection abuse
HANDSHAKE (handshake + PSH+ACK)	Real 3-way handshakes, each followed by PSH+ACK data with varying payload lengths, then close or abandon, repeated at a high connection rate	Labeled HANDSHAKE when a source shows 5 or more differing lengths in 32 packets. Shares handler and counter with SYN+ACK (H7); exclusive with STOMP (N2); label is attacker-controlled (N3)	Per source: handshakes completed per second, share of new flows whose first data packet arrives within a few ms, short flow lifetime, few packets per flow. Varying payload length as a supporting label	Per-source new-connection bucket on SYN (applied with the SYN-cookie path); per-/24 aggregate; blacklist once the bucket is exceeded	Yes (source completed a handshake)

Attack	What it is	Current gap (0.4.0)	Detection signal	Mitigation	Blacklist source?
SOCKET	Connection exhaustion: real connections held open idle, with zero window (sockstress), or slow-read, to consume sockets and memory	0.4.0 defines SOCKET as unusual-flag rate limiting (now configurable); connection count is kept in userspace via conntrack, with lag	Concurrent connections per source and per /24; connections idle with no payload; zero-window advertisements	Concurrent cap enforced in XDP from a counter map; reap idle sockets	Yes (source completed a handshake)
SYN-ACK (new class)	Inbound SYN+ACK flood, usually reflection: spoofed SYNs to third-party servers make them send SYN-ACKs to you	Still dispatched to key 8 with variable-length PSH+ACK, and counted as HANDSHAKE (H7)	SYN-ACK with no matching outbound SYN (TC egress records `{4-tuple, seq}`, XDP checks `ack_seq - 1`)	Drop unmatched SYN-ACKs	No (reflectors are innocent)

Telling STOMP and HANDSHAKE apart

Both are PSH+ACK data, so the plan keeps 0.4.0's payload-length profile and adds flow shape from layer 4. Both labels are computed for every source before any routing.

Label	Computed	STOMP	HANDSHAKE
Payload-length profile (0.4.0)	Per source, rolling 32-packet window, re-evaluated each window	One length (fewer than 5 differing)	5 or more differing lengths
Flow shape (layer 4)	Per flow and per source	High pps on few long-lived flows	High new-flow rate, few packets per flow, short lifetime

Rules:

- **Flow shape wins on disagreement.** Payload length is attacker-controlled; connection rate and flow lifetime cost the attacker real handshakes.
- **Labels drive reporting, not enforcement.** Both labels get the same enforcement (drop out-of-state, per-flow and per-source buckets on validated flows), so randomizing lengths gains the attacker nothing.

- **Both modes can be active at once.** Each label has its own counter, incremented at classification time before routing, so a combined or switching STOMP + HANDSHAKE attack stays visible (fixes N2).
- **Data, not the PSH bit.** Any TCP segment with payload > 0 is `DATA`; clearing PSH does not evade either label (fixes C3 remainder).
- **Gate on abnormal data, not total PSH+ACK.** Protection starts on out-of-state data or on per-flow/per-source rates above the destination baseline, never on host-wide PSH+ACK volume (fixes N1).

The `flows` map records first-seen time, handshake-completion time, packet and byte counts, and the first payload length plus a differing-length count per flow; the per-source aggregate records new flows per second.

Packet classification

Every packet gets exactly one class, decided in order, so counters never double-count and no packet falls through unclassified. Attack labels that depend on flow shape (STOMP, HANDSHAKE, SOCKET) are assigned per flow and per source by layer 4, on top of the packet class.

Order	Packet	Class	Validity test
1	Fails stage 0 (bad header, invalid flags, land, bogon, own-prefix source not in whitelist, disallowed protocol)	`SANITY`	None; always dropped
2	IPv4/IPv6 fragment	`FRAG`	TCP fragments invalid; others rate-checked per dst
3	TCP SYN, no ACK	`SYN`	SYN cookie reply in Attack
4	TCP SYN+ACK	`SYNACK`	Matches an outbound SYN
5	TCP RST or RST+ACK	`RST`	Socket lookup hit, sequence in window
6	Any other TCP segment with payload > 0, whatever the PSH, FIN or URG bits	`DATA`	Socket lookup hit; out-of-state = STOMP invalid. Valid packets feed layer 4 for STOMP, HANDSHAKE and SOCKET
7	TCP FIN or FIN+ACK, no payload	`FIN`	Socket lookup hit
8	TCP ACK, no payload	`ACK`	Socket lookup hit, or valid SYN cookie

Order	Packet	Class	Validity test
9	UDP	`UDP`	Port allowlist; not from a reflection port to a non-client port; QUIC header checks on 443
10	ICMP / ICMPv6	`ICMP`	Type allowlist; echo-reply matches a request
11	GRE	`GRE`	Peer allowlist
12	Anything else allowed by policy	`OTHER`	Per-dst rate only

Additional attacks to cover

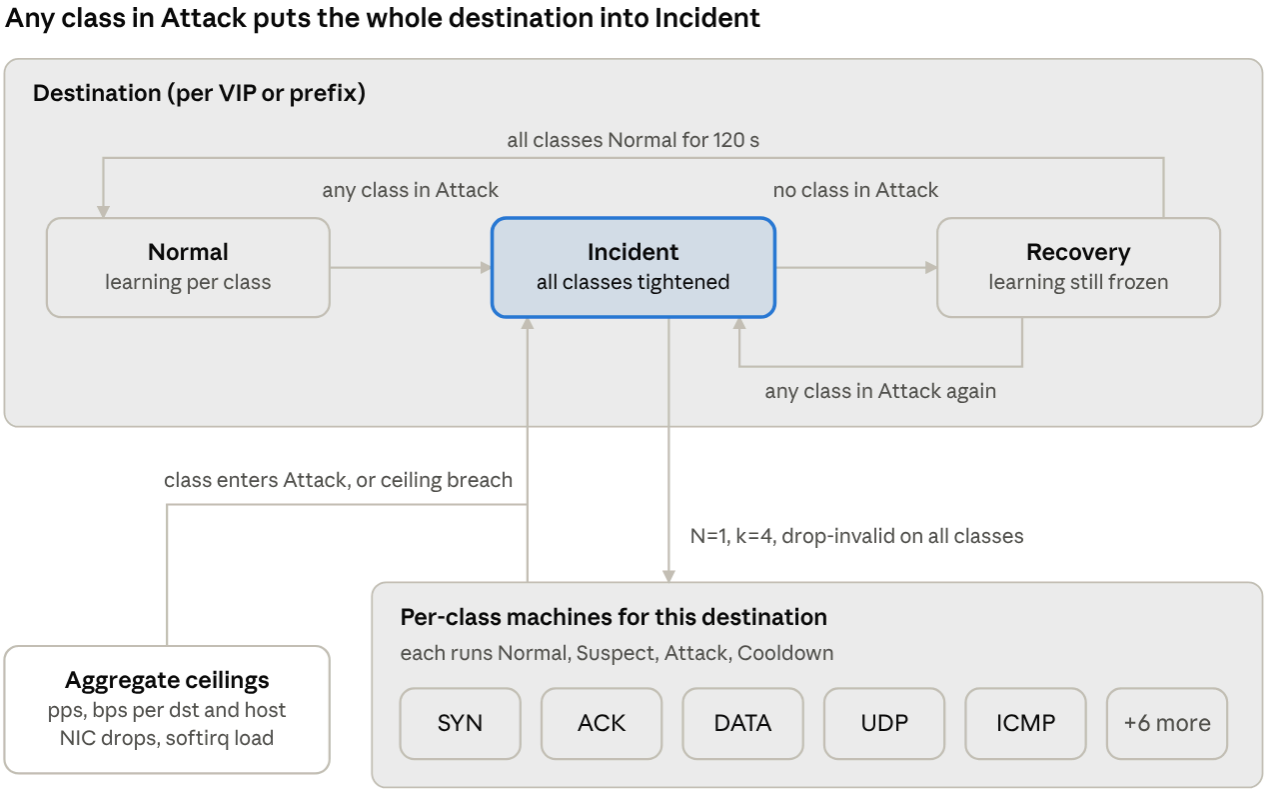
Attack	Signal	Mitigation
TCP flag anomalies (NULL, XMAS, SYN+FIN, SYN+RST, FIN-only, URG-only)	Flag combination not in the valid table	Stage 0 drop
RST, FIN, RST+ACK, FIN+ACK floods	Socket lookup miss	Drop invalid in Attack
SYN with payload / options abuse	SYN carrying data without a TFO cookie; missing MSS; oversized SYN	Stage 0 drop
IP fragment floods (UDP/ICMP fragments, tiny or overlapping)	Fragment rate per dst; first fragments under 576 bytes	Drop TCP fragments always; per-dst fragment bucket
IPv6 variants of every attack above	Same signals over IPv6	Parse IPv6 + extension headers (C5)
Bogon, own-prefix and land sources	Source in bogon list, inside your prefixes, or equal to dst	Stage 0 drop
Unknown IP protocol floods (proto 0, 255, unused ESP, SCTP, IPIP)	Protocol not in the destination's allowlist	Stage 0 drop
QUIC / UDP-443 floods	Invalid long-header version; Initial below 1200 bytes; short-header packets with unknown connection IDs	Header checks in XDP; QUIC Retry in the application as its cookie

Attack	Signal	Mitigation
Carpet bombing across a prefix	Per-prefix aggregate above baseline while each IP stays under its own	Per-prefix counter and mode
Pulse-wave	Repeated short bursts	Fast re-entry from Cooldown (lifecycle)

The last column of the required table is the key policy change: blacklisting is reserved for sources that completed a handshake, which removes the spoofed-blacklisting attack (H1).

Combined (multi-vector) attacks

Each destination gets an Incident state above its per-class state machines. Any class entering Attack, or any aggregate ceiling breach, puts the destination in Incident, which tightens detection for every other class and turns on drop-invalid across all of them.



two-level detection · destination incident over per-class machines

The per-class machines from the lifecycle section are unchanged; Incident only changes their parameters and keeps the destination protected between vectors.

What Incident changes

Setting	Normal	Incident and Recovery
Confirmation windows (N) for other classes	3	1
Sigma multiplier (k) / mean multiplier (m)	6 / 3	4 / 2
Baseline learning	Per class	Frozen for every class of the destination
Mode 1 drop-invalid	Only classes in Attack	Every class of the destination
Per-source buckets	Off	Classes in Attack only
Layer 4 flow tracking	Sampled	Every validated flow

Drop-invalid is safe to apply to all classes because it only removes packets that fail validity tests; a vector switch lands on mitigation that is already active.

Combined patterns and countermeasures

Pattern	Example	Why per-class detection misses it	Countermeasure
Sub-threshold blend	UDP + ACK + STOMP + ICMP, each at 70% of its threshold	No class crosses its own threshold	Per-destination aggregate pps/bps ceiling (layer 1) triggers Incident
Bandwidth-heavy amplification	DNS/NTP/memcached reflection with large packets	Packet rate looks moderate	bps ceilings per destination and per host; upstream escalation above 80% of link
Vector hopping	SYN, then ACK, then STOMP, then UDP, every 15 s	Each new class needs 3 s from cold start	Incident lowers N to 1 and keeps drop-invalid on for all classes

Pattern	Example	Why per-class detection misses it	Countermeasure
Decoy plus quiet vector	UDP amplification masking HANDSHAKE or SOCKET	XDP CPU saturates; NIC drops before XDP	NIC drop and softirq signals; cost-ordered pipeline keeps cheap drops first
State eviction	Random ACK flood alongside STOMP	Spoofed packets evict real flow state from LRU maps	Validate before creating state: only validated packets may insert into `flows` or per-source maps
Cross-class baseline drift	Slow ramp on UDP while SYN is mitigated	Only the attacked class freezes learning	Incident freezes learning for all classes of the destination
Spoofed plus real bots	Spoofed SYN flood with real-bot HANDSHAKE	Blocking on SYN volume hits innocent spoofed addresses	Order: SYN cookies first, then per-source buckets only for sources that passed cookies
Carpet bombing plus pulse	Rotating across a /24 in 10 s pulses	Per-IP baselines and cooldowns reset	Per-prefix aggregate with its own Incident; fast re-entry
Fragment plus UDP	Fragmented UDP flood to bypass port checks	Non-first fragments carry no ports	`FRAG` class with per-destination rate; reassembly not attempted

Incident record

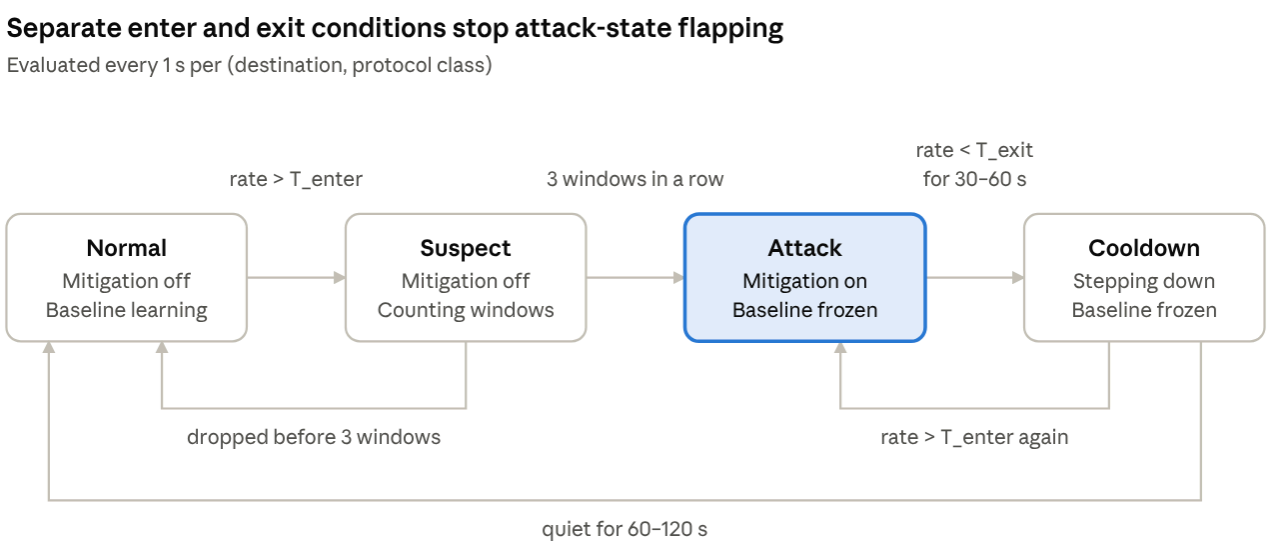
One record per destination incident, not one event per class: start and end time, peak pps and bps, the vector mix (each class's share of packets and bytes, and its invalid share), the classes that reached Attack, the highest mitigation mode, top source prefixes for blacklistable classes, and whether upstream escalation fired. A class is listed as a vector when it reached Attack or carried at least 5% of the destination's traffic during the incident.

Combined-attack test matrix

Scenario	Expected result
Spoofed SYN + real-bot HANDSHAKE	SYN cookies absorb spoofed SYNs; only bot sources are blacklisted
UDP amplification decoy + SOCKET	Both vectors in one incident record; SOCKET cap engages despite UDP volume
Random ACK + STOMP	STOMP still detected; `flows` entries for real clients are not evicted
Vector hopping every 15 s across 5 classes	Each new vector mitigated within 1 s; one continuous incident
Every class at 70% of threshold	Aggregate ceiling triggers Incident within 3 s
Carpet bombing a /24 in 10 s pulses	Prefix-level incident; no per-IP flapping
Mixed vectors at 110% of link capacity	Upstream escalation fires; host stays responsive

Attack lifecycle: start and end

An attack starts after 3 consecutive 1 s windows above `T_enter`, and ends only after the rate stays below a lower `T_exit` for 30–60 s plus a cooldown. A single threshold for both directions causes flapping.



attack lifecycle · 4 states, 6 transitions

Only Attack and Cooldown enforce mitigation; Cooldown re-enters Attack immediately, without waiting 3 windows.

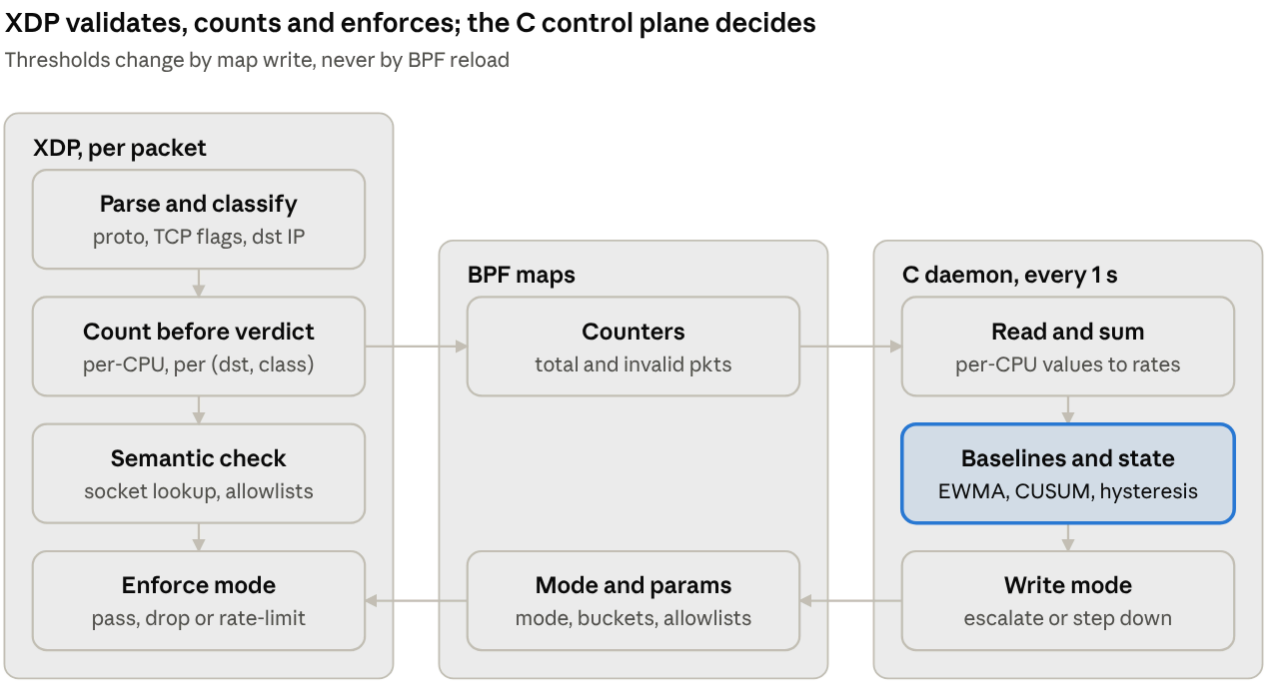
This machine runs once per (destination, class). For combined attacks it sits under a destination-level Incident state (see Combined attacks), which changes N, k and m for every class of the destination while any of them is in Attack.

Rules the state machine must follow:

- **Count before the verdict.** Increment counters at XDP entry, before `XDP_DROP`. Counting only passed traffic makes mitigation hide the attack, which ends ATTACK early and causes oscillation.
- **Freeze learning outside Normal.** Baselines update only in Normal; otherwise the attack raises its own threshold.
- **Hysteresis gap.** `T_exit = max(1.5 x mean, 0.5 x T_enter)`, always well below `T_enter`.
- **Fast re-entry.** Pulse attacks (on/off cycles) stay in Attack/Cooldown instead of restarting detection each pulse.
- **One machine per key.** Each (destination, protocol class) has its own state; a capacity-ceiling breach can force Attack on all keys of that host.
- **Event log.** Record start, end, peak pps, top sources and mitigation mode reached, for tuning and customer reports.

Implementation split: XDP and C control plane

The XDP program only counts and enforces; the existing C daemon owns every statistic and decision, in one control loop that replaces both attack workers in ebpf_firewall_core.c. Tuning k, m, N or M is a map write, with no verifier work or program reload.



data plane and control plane · XDP, maps, C daemon

Counting happens before the semantic check and the verdict, so the daemon always sees the full offered load.

Map layout

Map	Type	Key	Value	Written by
`counters`	`LRU_PERCPU_HASH`	`{dst, class}`	`{pkts, bytes, invalid_pkts}`	XDP
`mode`	`LRU_HASH`	`{dst, class}`	`{mode, expires_ns}`	Go
`params`	`ARRAY`	`class`	`{bucket_rate, bucket_burst, floors}`	Go
`udp_ports`, `gre_peers`	`HASH`	`{dst, port}` / peer IP	allow flag	Go
`src_buckets`	`LRU_HASH`	`{src prefix, class}`	token-bucket state	XDP

```
struct det_key {
    __u32 dst[4];   /* IPv4-mapped IPv6 */
    __u8  class;    /* ACK, SYN, SYNACK, RST, ICMP, UDP, GRE, STOMP,
HANDSHAKE, SOCKET */
    __u8  pad[3];
};
```

`expires_ns` in the `mode` map is the watchdog: if the daemon stops refreshing it, XDP falls back to a conservative static profile instead of holding a stale mode forever.

XDP pipeline order

1. Parse Ethernet, 802.1Q/QinQ, IPv4 or IPv6 (C5).
2. Count `{dst, class}` and bytes, before any verdict (H2). Whitelisted traffic is counted too (M6).
3. Stage 0 sanity drops, each with its own reason counter.
4. `blocked_ips` check (proven sources only).
5. Validity test per class: socket lookup, SYN cookie, outbound-SYN match, port/peer/protocol allowlists. Increment the invalid counter on failure.
6. Read `mode` for `{dst, class}`; apply drop-invalid, per-flow, per-source or per-dst buckets.
7. Geo policy, as today.

Two rules keep this pipeline correct under combined attacks:

- **Cost order.** Steps 3–5 run cheapest first: header checks and flag table, then hash lookups (`blocked_ips`, port, peer and protocol allowlists), and only then socket lookups and SYN-cookie math. A UDP decoy flood is dropped before it can spend CPU on TCP validation.
- **Validate before state.** Only packets that passed step 5 may insert into `flows`, `src_buckets` or the connection counter. Invalid packets may only increment counters, so a spoofed flood cannot evict real flow state from LRU maps.

The daemon adds an `incident` map keyed by destination (and by prefix for carpet bombing) holding `{state, since_ns, n_override, k_override}`. XDP reads only the effective `mode` per `{dst, class}`; the daemon folds the Incident overrides into it.

Migration from current maps

Current	Problem	Replacement
`global_*_pkt_counter` (9 single-key `PERCPU_ARRAY`)	Global only, read-and-zeroed, counted after verdict	One `counters` `LRU_PERCPU_HASH` keyed `{dst, class}`, monotonic
`*_counter_fixed`, `*_counter_fixed_metrics`, `burst_*_state` (24 per-source LRU maps)	Run outside Attack; drive 1 h blocks from spoofable packets	One `src_buckets` LRU per family, consulted only in Attack
`tcp_established_map` fed from conntrack	`(src_ip, src_port)` key, reply-tuple bypass, loose pickup	`bpf_skc_lookup_tcp()` for validity; `flows` LRU keyed by 4-tuple for layer 4 rates
`syn_challenge`, `syn_validated_ips`	Bypassable, adds latency	Removed; XDP SYN cookies
`tcp_connection_map` (counted in userspace)	Event lag; 200 per IP hurts CGNAT	Counter map read by XDP on SYN; cap per source and per /24, Attack only
`g_attack_stats` in `.bss`, written whole by two threads	Race, config overwrite (C1)	`mode` map per `{dst, class}`, written by one thread
`blocked_ips`	Filled by spoofed sources	Kept, but written only for proven sources, with reason code

Maps added in 0.4.0:

0.4.0 map or program	Keep?	Becomes
`xdp_parse_psh_ack` (key 6)	Yes, reworked	The `DATA` classifier: computes both labels and counts them before routing; matches on payload > 0 instead of the PSH bit
`psh_ack_len_profile` (per source)	Yes	Length-profile label; rolling window instead of a permanent `klass`; also kept per flow in `flows`

0.4.0 map or program	Keep?	Becomes
`psh_ack_rate` (per source)	Replaced	Per-source data rate inside the layer 4 aggregate, compared with the destination baseline
`global_psh_ack_filtered`	Folded in	One drop-reason counter in `counters` (`{dst, class, reason}`)
`xdp_parse_stomp` (key 9), `xdp_parse_handshake` (key 8)	Merged	One enforcement path for validated data flows; SYN+ACK moves to a new key 10
`g_socket_config`	Yes	Stage 0 flag-anomaly limits (NULL, XMAS, FIN-only, odd combos)

Kernel requirements

`bpf_skc_lookup_tcp` in XDP needs 5.2+, `bpf_tcp_raw_gen_syncookie_ipv4/ipv6` needs 6.0+, and `bpf_sock_destroy` needs 6.5+. Open question: which kernel runs on your production nodes? On older kernels, keep kernel `tcp_syncookies` and close sockets from userspace.

Hardware: NIC speed and derived parameters

Capacity-dependent parameters are derived from the NIC each node actually has, because Dorian nodes range from 1 Gb/s community servers to 400 Gb/s edges. Above roughly 25 Gb/s the CPU, not the link, is usually the first limit, so the derivation uses the smaller of what the link can deliver and what the cores can process.

Link speed	64 B frames	512 B frames	1500 B frames
1 Gb/s	1.49 Mpps	0.23 Mpps	0.08 Mpps
10 Gb/s	14.9 Mpps	2.35 Mpps	0.82 Mpps
25 Gb/s	37.2 Mpps	5.87 Mpps	2.06 Mpps
40 Gb/s	59.5 Mpps	9.40 Mpps	3.29 Mpps
100 Gb/s	148.8 Mpps	23.5 Mpps	8.22 Mpps
200 Gb/s	297.6 Mpps	47.0 Mpps	16.4 Mpps

Link speed	64 B frames	512 B frames	1500 B frames
400 Gb/s	595.2 Mpps	94.0 Mpps	32.9 Mpps

Line rate per direction; values include the 20 bytes of preamble and inter-frame gap per frame.

Deriving parameters from the NIC

In 0.4.0 the panel backend computes these values from facts the node reports and writes them into `firewall.config`. In the redesign the daemon computes them itself and reports them to the panel.

1. Detect at daemon start and on every link change (netlink `RTM_NEWLINK`):

Input	Source	Notes
Link speed	`/sys/class/net/<dev>/speed` (Mb/s)	Bond: sum of up slaves in 802.3ad and balance modes, one slave in active-backup (`bonding/mode`, `bonding/slaves`). VLAN: read the lower device. `-1` or unknown (some virtual NICs): admin-entered value, else 10 Gb/s with a panel warning
Driver	`/sys/class/net/<dev>/device/driver` or `ethtool -i`	Decides attach mode and hardware tier
RX queues	`ethtool -l` (combined)	Queues XDP actually runs on
Cores on the NIC's NUMA node	`/sys/class/net/<dev>/device/numa_node`, then `/sys/devices/system/node/node<N>/cpulist`	
Per-core pipeline rate	Load test; until then 3 Mpps per core for the full pipeline (approximate)	Admin enters the measured value in the panel
Transit capacity	Panel (admin)	Smallest upstream port; defaults to link speed

2. Compute:

$$\text{line_pps} = \frac{\text{link_bps}}{(64 + 20) \times 8}$$

$$\text{cpu_pps} = \min(\text{rx_queues}, \text{numa_cores}) \times \text{per_core_pps}$$

$$\text{host_ceiling_pps} = 0.65 \times \min(\text{line_pps}, \text{cpu_pps}) \qquad \text{host_ceiling_bps} = 0.8 \times \min(\text{link_bps}, \text{transit_bps})$$

$$\text{cap}_c = f_c \times \text{host_ceiling_pps} \qquad \text{gate}_c = \mathrm{clamp}(3 \times p99_c,\ \text{floor}_c,\ \text{cap}_c)$$

During the first 7 days, before a weekly p99 exists, the gate starts at cap / 4 (still within floor and cap).

3. Class fractions and floors:

Class (0.4.0 key)	Fraction of host ceiling (cap)	Floor (pps)
SYN (`syn_threshold`)	2%	20,000
ACK (`ack_threshold`)	5%	50,000
RST (`rst_threshold`)	1%	10,000
ICMP (`icmp_threshold`)	0.5%	2,000
UDP (`udp_threshold`)	5%; 15% with HTTP/3	20,000; 50,000 with HTTP/3
GRE (`gre_threshold`)	0.5% with tunnels; floor only without	1,000
PSH+ACK gate (`stomp_threshold` = `handshake_threshold`)	10%	50,000

4. Examples (3 Mpps per core, transit at least link speed; start gate / cap):

	10 Gb/s, 8 cores	25 Gb/s, 16 cores	100 Gb/s, 32 cores	200 Gb/s, 64 cores
Binding limit	Link	Link	CPU	CPU
Host ceiling	9.7 Mpps, 8 Gb/s	24.2 Mpps, 20 Gb/s	62.4 Mpps, 80 Gb/s	124.8 Mpps, 160 Gb/s
`syn_threshold`	48k / 193k	121k / 484k	312k / 1.25M	624k / 2.50M
`ack_threshold`	121k / 484k	302k / 1.21M	780k / 3.12M	1.56M / 6.24M
`rst_threshold`	24k / 97k	60k / 242k	156k / 624k	312k / 1.25M
`icmp_threshold`	12k / 48k	30k / 121k	78k / 312k	156k / 624k
`udp_threshold` (HTTP/3 on)	363k / 1.45M	907k / 3.63M	2.34M / 9.36M	4.68M / 18.7M
PSH+ACK gate	242k / 967k	605k / 2.42M	1.56M / 6.24M	3.12M / 12.5M

	10 Gb/s, 8 cores	25 Gb/s, 16 cores	100 Gb/s, 32 cores	200 Gb/s, 64 cores
`stats_detail_sample_divisor`	1	1	4	8

5. What does not scale with NIC speed. Per-source limits (fixed windows, connection caps, buckets), block durations, protection and cooldown durations, and sensitivity multipliers describe client behavior, not host capacity, so they stay as in the panel defaults. Attach mode and the hardware tier depend on the driver, not the speed:

Driver	Attach mode	Hardware tier 0
`mlx5_core` (ConnectX)	Native	TC flower offload with drop and police
`ice`, `i40e`, `ixgbe` (Intel)	Native	n-tuple drop rules only
`virtio_net`, `ena` (virtual)	Native where supported	None
Unknown	Generic, with a panel warning	None

6. Rules:

- **Recompute on link change.** A bond losing a slave lowers ceilings and caps immediately; increases wait 60 s so a flapping link cannot loosen protection.
- **Floors and caps bind the admin too.** An admin override outside them is allowed only with a panel warning, and is logged.
- **Show the inputs.** The node reports detected speed, driver, queues, NUMA cores and derived values; the panel shows them read-only next to the Tier 2 fields.
- **Redesign:** host ceilings feed layer 1 and the Incident aggregate; per-class gates become learned baselines, still bounded by the same caps.

What this changes in the plan

- **Capacity ceiling = measured pipeline throughput.** Load-test the full XDP pipeline (stage 0, lookups, socket lookup) across all RX queues with TRex or pktgen at 64 B and 512 B per flood type. Native XDP drop rates are roughly 10–25 Mpps per core for simple drops and lower with map and socket lookups (approximate; measure on your CPUs). The measured per-core rate replaces the 3 Mpps estimate in the derivation above; the ceiling is 65% of the binding limit.

- **Per-queue signals.** Floods with one or a few fixed 5-tuples hash to one RX queue and saturate one core while the rest idle. The daemon reads per-queue drop counters (`ethtool -S`, e.g. `rx<N>_xdp_drop`, `rx<N>_buff_alloc_err` on mlx5) and per-core softirq time; a single hot queue counts as a ceiling breach for the destinations it carries.
- **NIC hardware filters become tier 0 now.** Above the XDP ceiling, the only host-side defense is dropping in NIC hardware before packets reach the CPU (next subsection).
- **Uplink, not NIC, sets escalation.** Upstream escalation triggers use the smaller of the NIC speed and the transit port capacity.

Tier 0 on ConnectX NICs

Applies to nodes whose driver is mlx5_core (NVIDIA ConnectX-6 Dx, ConnectX-7 and similar); other drivers get the reduced tier from the driver table above. ConnectX supports TC flower hardware offload (`skip_sw`) with drop and police actions, matching on IP protocol, addresses and prefixes, L4 ports and TCP flags, plus `ethtool -N` n-tuple drop rules.

Use in hardware	Example rule	Source of the rule
Reflection source ports to the attacked VIP	UDP, src port 123, dst VIP: drop	Incident vector mix
Disallowed IP protocols	proto 47 (GRE) to VIPs without tunnels: drop	Static policy
Invalid TCP flag combinations	flags SYN+FIN: drop	Static stage 0 policy
Per-destination class cap (mode 3)	UDP to dst VIP: police at the ceiling rate	Mode 3 from the state machine
Proven-source and prefix blocks	src /24 of a STOMP botnet: drop	Blacklist policy

Rules for this tier are the same as for BlueField-3 below: coarse matches only, never per-IP rules for spoofable sources; hardware drop counters (`tc -s filter show`) are added to the XDP counters so count-before-verdict holds; and each rule is withdrawn with its incident. The daemon emits one abstract rule set with a TC-flower backend today and a DOCA Flow backend later, so the move to BlueField-3 changes only the backend.

Host tuning for high-speed NICs

- XDP in native (driver) mode on mlx5; never generic mode.

- RX queues equal to physical cores on the NIC's NUMA node; IRQs pinned one per core; `irqbalance` disabled for those IRQs.
- Maximum RX ring size; daemon and maps on the same NUMA node as the NIC.
- `flows` as `LRU_PERCPU_HASH` is safe because RSS keeps a flow on one CPU; per-source aggregates span CPUs, so the daemon sums them and pushes per-source verdicts back.

Open question: which exact NIC model and firmware do you run? It decides how many hardware rules and meters are available.

Future: BlueField-3 hardware offload

With BlueField-3, every check that needs no host socket state moves into NIC hardware, so floods of those classes are dropped before they cross PCIe or use host CPU. Host XDP keeps only the checks that need sockets or per-flow state.

What the hardware provides: DOCA Flow builds packet-processing pipes in NIC hardware, runs either on the host or on the DPU, matches up to two packet layers including tunnels, and has a monitor stage that can count, meter or mirror (DOCA Flow guide). Hardware-Managed Flow Steering gives a much higher rule insertion rate than software-managed steering, and BlueField-3 requires the DOCA-Host driver because MLNX_OFED does not support it (DOCA 3.2 changes).

Tier	Runs on	Handles	Why there
0. NIC hardware	DOCA Flow pipes on the BlueField-3 eSwitch	Stage 0 checks that are plain matches (invalid flag combos, TCP fragments, disallowed IP protocols, bogon and own-prefix sources); port, peer, protocol and ICMP-type allowlists; blocklist of proven sources and prefixes; per-destination and per-class meters (mode 3 hard cap); per-`{dst, class}` counters	Line rate with zero host CPU; a decoy flood cannot starve XDP

Tier	Runs on	Handles	Why there
1. Host XDP	mlx5 native XDP on the host	Socket-lookup validity (ACK, RST, FIN, DATA); SYN cookies via `XDP_TX`; SYN-ACK matching with TC egress; layer 4 flows (STOMP, HANDSHAKE, SOCKET); per-source buckets	Needs the host socket table and per-flow state
2. Control	C daemon on the host, plus a small agent on the DPU Arm	Baselines, per-class and Incident state machines, rule programming for both tiers, upstream escalation	Daemon reads XDP maps directly; the DPU agent only programs pipes and exports hardware counters

Rules for the hardware tier:

- **Hardware counters are part of count-before-verdict.** Tier 0 drops never reach XDP, so the daemon sums hardware counters and XDP counters per `{dst, class}`. Without this, offloading a class would hide its attack and end Incident early (the H2 bug again).
- **No per-IP rules for spoofable sources in hardware.** Spoofed floods would churn rule insertion and fill tables. Hardware holds allowlists, prefixes, proven-source blocks and meters only.
- **Meters replace XDP token buckets for per-destination caps.** Per-source buckets stay in XDP, because only validated sources get them.
- **NIC speed is not uplink capacity.** A 2x200 Gb/s card does not help if the transit port is smaller; upstream escalation thresholds use the transit port capacity.

Open question: will the card run in DPU mode (eSwitch controlled from the Arm cores) or NIC mode? DPU mode suits the tier split above; in NIC mode the DOCA Flow program runs on the host instead of the Arm agent.

Graded mitigation modes

Mitigation escalates in steps rather than switching between pass-all and drop-all. Cheap false positives allow aggressive detection thresholds.

Mode	Entered when	XDP action	Effect on legitimate traffic
0 Observe	NORMAL	Count only	None
1 Drop invalid	ATTACK, semantic layer fired	Drop packets failing the semantic check (no socket, bad cookie, unknown peer)	None expected
2 Rate-limit sources	ATTACK persists, or volume layer fired with valid-looking traffic	Per-source token bucket (per /32, then /24)	Heavy legitimate clients may be throttled
3 Hard cap	Capacity ceiling exceeded	Global per-dst token bucket at the ceiling	Some legitimate loss; protects the host

Escalate one step after N windows in the current mode without the rate falling below `T_enter`. De-escalate one step per cooldown period, never straight to mode 0.

Per-class specifics inside mode 1:

- **SYN:** answer SYNs with XDP-generated SYN cookies (`XDP_TX`) and validate the returning ACK with `bpf_tcp_check_syncookie()`. This replaces the retransmit challenge and costs legitimate clients no extra round trip.
- **HANDSHAKE:** per-source new-connection bucket on SYN; a source that exceeds it after completing handshakes is blacklisted. SYN-ACK: drop SYN-ACKs with no recorded outbound SYN.
- **UDP / GRE / ICMP:** drop on allowlist failure (port, peer, type).

Blacklist policy

Evidence	Spoofable?	Action
Invalid ACK/RST/FIN, UDP to closed port, unknown GRE peer, ICMP, unmatched SYN-ACK, SYN without cookie reply	Yes	Drop packet; never add to `blocked_ips`
Completed handshake or valid cookie, then flow/source rate abuse (STOMP, HANDSHAKE, SOCKET)	No	Add to `blocked_ips` with reason and expiry
Operator or API block	n/a	Add to `blocked_ips`

Block expiry should start short (60 s) and double on repeat offence, up to the configured `black_ip_duration`, so a wrongly blocked CGNAT address recovers quickly.

Upstream escalation

When host bps stays above 80% of link capacity for 10 s, or NIC drops keep rising while every class is already at its highest mode, the host cannot help itself. The daemon then escalates outward, in order of how much traffic it sacrifices:

1. **BGP Flowspec** rules built from the incident's vector mix (for example: UDP, source port 123, to the attacked VIP), announced via GoBGP or ExaBGP to the upstream.
2. **Scrubbing provider** diversion for the attacked prefix, if contracted.
3. **RTBH** (remotely triggered blackhole) for the single attacked /32 as a last resort, which sacrifices that VIP to save the rest of the host.

Each step is withdrawn after the incident ends plus its cooldown. Open question: does your upstream accept Flowspec or RTBH communities?

Rollout, testing and validation

Roll out in five phases, each gated on measured results before the next begins.

Phase 0 — quick fixes on 0.4.0 (days, not weeks): C1 is done. Remaining: set `nf_conntrack_tcp_loose = 0` (C4); stop inserting `key_repl` into `tcp_established_map` (C2, short-term); switch attack counters to the same delta method the traffic-stats worker now uses (H3); count PSH+ACK labels before routing in `xdp_parse_psh_ack` so both STOMP and HANDSHAKE counters stay live (N2); move SYN+ACK to its own counter (H7); raise `stomp_threshold` and the per-source PSH+ACK window above measured legitimate peak until layer 4 exists (N1). Gate: no flapping under a replayed SYN flood, and no legitimate client blocked during a replayed peak-hour PSH+ACK load.

1. **Fix and instrument.** Apply the define fixes. Add per-CPU counters keyed by (dst, class), counted before the verdict, plus invalid-traffic counters. Export to Prometheus or similar.
   - Gate: counters match an independent capture (tcpdump or NIC stats) within 1%.
2. **Capacity profiling.** Load-test each hardware class with trafgen, pktgen or MoonGen for each flood type. Record the pps knee where p99 latency or CPU saturates.
   - Gate: a ceiling value per hardware class and flood type.
3. **Shadow mode.** Run the new C control loop with baselines and state machine, logging decisions without enforcing. Keep the old static thresholds enforcing.

    - Gate: 7+ days of baselines; zero ATTACK states during known-clean periods; replayed attack captures detected.
4. **Enforce mode 1 only.** Enable drop-invalid mitigation. It is low risk because it drops only traffic failing semantic checks.
    - Gate: no customer-visible errors; false-positive rate below target.
5. **Enable modes 2–3** and retire the static thresholds.

Test cases

- Sudden flood at 5x, 50x and above capacity, per protocol.
- Slow ramp: 1.5x per 10 minutes over two hours (tests growth cap and CUSUM).
- Pulse or carpet attack: 10 s on, 20 s off (tests cooldown and fast re-entry).
- Legitimate surge: flash crowd with valid handshakes (must not reach mode 2).
- Attack during mitigation: confirm detection still sees pre-drop rates.

Attack-specific cases for the required set:

- **STOMP:** open real connections, then flood PSH+ACK with valid sequence numbers; layer 4 must trigger and blacklist only those sources.
- **SOCKET:** hold 10K idle connections from 50 sources, and separately run connect/close churn; verify caps and idle reaping.
- **HANDSHAKE:** 1,000 sources each completing 50 handshakes per second, sending PSH+ACK data, then closing; layer 4 must trigger on new-flow rate and blacklist only those sources. SYN-ACK: reflect SYN-ACKs from public servers while the CDN keeps fetching from origins; origin SYN-ACKs must pass.
- **Spoofed blacklisting:** ACK and UDP floods spoofing a test client's IP; that client must never enter `blocked_ips`.
- **Bypass regressions:** packets spoofed from `server_ip:443` and from an origin's `ip:port` (C2); IPv6 and VLAN-tagged floods (C5); SYN+FIN combos (H5).
- **QUIC:** real HTTP/3 clients at 2K pps each stay unthrottled outside Attack.

Regression cases for the 0.4.0 PSH+ACK logic:

- **PSH bit cleared:** STOMP and HANDSHAKE floods sent as plain ACK with payload must still be labeled and mitigated.
- **Legitimate peak:** replay peak-hour client traffic (uploads, WebSockets, many HTTP/2 requests) at 2x; protection must not start and no client is blocked.
- **Label flipping:** a STOMP flood with randomized payload lengths, and a HANDSHAKE flood with fixed lengths; flow shape must still label them correctly.
- **STOMP + HANDSHAKE together:** both labels active in one incident record.

- **SYN-ACK during HANDSHAKE:** reflected SYN-ACKs must not change the HANDSHAKE count or mode.

Metrics to track

- Time to detect (target under 5 s) and time to declare end.
- False positives per week per destination.
- Number of ATTACK/NORMAL flaps per incident (target 0).
- Legitimate traffic dropped during mitigation, estimated from valid-flow drops.

Combined-attack metrics:

- Time from a vector switch to that vector's mitigation (target under 1 s during Incident).
- Vector-mix accuracy: classes listed in the incident record versus the classes actually injected in tests.
- Flow-state survival: share of real-client `flows` entries still present after a spoofed flood (target above 99%).

Initial parameter defaults and risks

Start from these values in shadow mode and tune from observed false positives and detection times.

Existing `firewall.config` keys carry over so operators keep one file:

Current key	New meaning
`<class>_threshold` (e.g. `syn_threshold = 100000`)	Capacity-ceiling override per class; adaptive baseline decides below it
`<class>_burst_pkt`, `<class>_burst_count_per_sec`	Per-source bucket burst and rate, Attack only
`<class>_fixed_threshold`, `<class>_fixed_check_duration`	Removed; replaced by the per-source bucket
`<class>_protection_duration` (120 s)	Cooldown length
`black_ip_duration` (3600 s)	Upper bound of the escalating block expiry
`tcp_connection_limit_cnt` (200)	SOCKET concurrent cap per source in Attack; add a per-/24 cap
`challenge_timeout`	Removed with the retransmit challenge

0.4.0 keys:

0.4.0 key	New meaning
`stomp_threshold = 10000` (host-wide PSH+ACK gate)	Removed as a gate; data protection starts on abnormal data per destination (N1). Kept only as a capacity-ceiling override for the `DATA` class
`stomp_fixed_threshold`, `handshake_fixed_threshold = 200` per 10 s	Per-source data bucket for validated flows, sized from the destination baseline
`handshake_threshold`	HANDSHAKE label ceiling override; no longer counts SYN+ACK
`socket_fin_threshold`, `socket_fixed_threshold`, `socket_fixed_check_duration`	Stage 0 flag-anomaly limits; the SOCKET attack class becomes connection exhaustion

New parameters for combined attacks:

Parameter	Default	Notes
Incident N / k / m	1 / 4 / 2	Applied to every class of the destination
Incident exit	All classes Normal for 120 s	Recovery state in between
Destination aggregate ceiling	Sum of class baselines x 3, capped by capacity	pps and bps
Host bps escalation trigger	80% of min(link, transit) for 10 s	Starts upstream escalation
NIC drop trigger	Any increase for 3 consecutive seconds	Treated as ceiling breach
Softirq trigger	Above 80% per core for 3 s	Treated as ceiling breach
Vector listing threshold	Reached Attack or at least 5% of traffic	For the incident record

Baseline, lifecycle and mitigation defaults

These match `firewall.redesign.config`; capacity-dependent values come from the Hardware section.

Parameter	Default	Config key
Sample interval	1 s	`sample_interval_ms`
EWMA half-life	45 min	`ewma_half_life_minutes`
k (sigma multiplier)	6 (4 in Incident)	`baseline_k`, `incident_k`
m (mean multiplier)	3 (2 in Incident)	`baseline_m`, `incident_m`
Gate floors and caps	Per class	`floor_*`, `cap_fraction_*`
Invalid-traffic trigger	1,000 pps and 5% of the class	`invalid_floor_pps`, `invalid_ratio`
N (enter windows)	3 (1 in Incident)	`confirm_windows`, `incident_confirm_windows`
M (exit windows)	45 s below `T_exit`	`exit_windows`
`T_exit`	max(1.5 x mean, 0.5 x `T_enter`)	`t_exit_mean_multiplier`, `t_exit_enter_fraction`
Cooldown	90 s	`cooldown_seconds`
Incident exit	All classes Normal for 120 s	`recovery_seconds`
Baseline growth cap	2x per hour	`baseline_growth_cap_per_hour`
Learning period	7 days; gates start at cap / 4	`learning_days`
Capacity ceiling	65% of min(64 B line rate, cores x per-core rate)	`ceiling_fraction`, `per_core_pps`
Block expiry	60 s, doubling per repeat, up to 3,600 s	`block_initial_seconds`, `block_max_seconds`

Risks

- **Baseline poisoning by slow ramps.** Mitigated by the growth cap, CUSUM and frozen learning during attacks.

- **Asymmetric routing.** Socket-lookup signals assume the server sees both directions. On anycast or DSR paths, validate the invalid-ACK rate on a clean day first.
- **Per-destination map size.** Many destinations times protocol classes can grow large; use LRU hash maps and aggregate cold destinations.
- **Control-plane failure.** If the C daemon stops, the XDP program must keep the last mode with a timeout back to a conservative static fallback, never to fail-open drop-all.
- **Carpet bombing across a prefix.** Per-destination baselines may each stay under threshold. Add a per-prefix aggregate counter.

Two risks specific to this codebase: XDP SYN cookies and socket lookups assume this host terminates the TCP connection (true for nginx/OpenResty on the same box, false if it forwards to backends), and removing the conntrack mirror changes how `tcp_connection_map` is fed, so ship the XDP-side counter before deleting the conntrack thread.

Dorian panel configuration

`firewall.config` is edited through the Dorian panel, in three tiers by who may change a key and what a wrong value can break. Because Dorian nodes can be contributed and run by customers, node owners get policy controls only; thresholds stay with the platform admin.

Tier 1: Basic (node owner, Servers → server → Firewall)

Policy choices shown as plain controls, never as raw pps values.

Panel control	Writes	Limits
Geo blocking mode and countries	`geo_check`, `geo_countries`	None
Block duration	`black_ip_duration`	300–7200 s
Connection limit per source IP	`tcp_connection_limit_check`, `tcp_connection_limit_cnt`	At least 200; warn below 1,000 for mobile and CGNAT audiences
"This server serves HTTP/3"	`udp_threshold`, `udp_fixed_threshold`	On sets the HTTP/3-safe values
"This server runs DNS or other UDP services"	Same UDP keys	Preset values

Panel control	Writes	Limits
"This server terminates GRE tunnels"	`gre_valid` and the `gre_*` thresholds	Off means any GRE is treated as an attack
Sensitivity per class: Low / Normal / High	Multiplies that class's thresholds (e.g. 2x / 1x / 0.5x)	The owner never types pps values; the result is still clamped to the class floor and cap

Tier 2: Advanced (platform admin only, per server or server group)

Wrong values here block real users or switch protection off. Next to each threshold the panel shows the detected NIC speed, driver, queues and cores, the derived floor and cap, the measured weekly p99 from InfluxDB, and the recommended value (3x p99 within floor and cap).

Keys	Notes
`syn_threshold`, `ack_threshold`, `rst_threshold`, `icmp_threshold`, `udp_threshold`, `gre_threshold`	Host-wide gate per class
`stomp_threshold` + `handshake_threshold`	Edited as one field, "PSH+ACK gate", because the daemon uses the lower of the two
`*_fixed_threshold`, `*_fixed_check_duration`	Per-source limits during an attack
`*_protection_duration`	Hold time after the attack ends
`*_valid` per class	Disabling a defense is an admin decision
`dev`	Prefilled by the install script; editable when the NIC changes
`stats_detail_sample_divisor`	Performance tuning on 200 Gb/s nodes
`socket_fin_threshold`, `socket_fixed_threshold`, `socket_fixed_check_duration`	Rarely changed

Tier 3: Not in the panel (installer or code)

Keys	Why
`log`, `geo_db_ipv4_path`, `geo_db_location_path`	Installation paths
`attach_mode`	A wrong value silently falls back to slow generic XDP or fails to attach
`*_burst_pkt`, `*_burst_count_per_sec`	No effect in 0.4.0: the burst gap is 1 s / `burst_count_per_sec`, so N bursts need more than the 1 s window and the detector never fires. A 0 stops the daemon. The panel writes fixed values
`challenge_timeout`	Removed in the redesign
`tcp_seg_check`	Always 1
`map_size_*`	Memory sizing tied to the hardware

Rules the panel backend enforces

1. **Version-aware schema.** 0.4.0 refuses to start on an unknown key. Each node reports its firewall version, and the panel generates the config from the schema for that version.
2. **Full file, every time.** Tier 3 values are filled from defaults; the panel never writes a partial file.
3. **Validation before push:** durations > 0; `burst_count_per_sec` ≠ 0; `stomp_threshold` = `handshake_threshold`; floors such as `udp_fixed_threshold` >= 30,000 per 10 s while HTTP/3 is on and `stomp_fixed_threshold` >= 5,000 per 10 s.
4. **Audit and rollback.** Each pushed version is stored with who changed what, so a bad change on a community node can be reverted from the panel.

Two more constraints from the 0.4.0 parser and from new nodes:

- **Lines under 128 bytes.** `load_config()` reads with `fgets` into a 128-byte buffer (`MAX_LINE_LEN`); a longer line is split and the remainder is parsed as a new line. Values are read with `%s`, so a trailing `# comment` after a value is safe.
- **Fallback profile.** Until a node reports its NIC facts, the panel writes the derived keys for 10 Gb/s, 8 cores, 3 Mpps per core, HTTP/3 on and no GRE tunnels, then rewrites them once the facts arrive.

Reference files: `firewall.config` is the 0.4.0 template with every derived key marked `[DERIVED]`; `firewall.redesign.config` is the target schema, with each key tagged `[T1]`, `[T2]` or `[T3]` for its panel tier and `auto` where the daemon detects or derives the value.

Phase 0 defaults the panel should ship (0.4.0)

Key	0.4.0 default	Panel default	Why
`stomp_threshold`, `handshake_threshold`	10,000	Derived from NIC and CPU (Hardware section)	PSH+ACK gate above legitimate client traffic (N1)
`stomp_fixed_threshold`, `handshake_fixed_threshold`	200 / 10 s	5,000 / 10 s	The window also applies to established clients during protection
`udp_threshold`	100,000	Derived from NIC and CPU (Hardware section)	Room for HTTP/3
`udp_fixed_threshold`	500 / 10 s	30,000 / 10 s	50 pps blocked every active HTTP/3 client
`rst_fixed_threshold`	100 / 10 s	500 / 10 s	Spoofable; low limits block the spoofed address
`icmp_fixed_threshold`	100 / 10 s	200 / 10 s	Monitoring and PMTU traffic
`syn_fixed_threshold`	150 / 3 s	300 / 3 s	CGNAT gateways and proxies
`tcp_connection_limit_cnt`	200	1,000	CGNAT
`black_ip_duration`	3,600 s	600 s	Limits damage from blacklisted spoofed addresses (H1)

"Measure" values follow the Hardware section: start at cap / 4, then 3x the node's weekly p99, always within the class floor and the NIC- and CPU-derived cap. syn_threshold, ack_threshold, rst_threshold, icmp_threshold and gre_threshold are derived the same way. Trade-off: raising `handshake_threshold` with the PSH+ACK gate means SYN+ACK floods below that rate no longer start protection until SYN-ACK gets its own handler (H7); the kernel still drops unmatched SYN-ACKs.

After the redesign

Most Tier 2 numbers disappear because thresholds are learned per destination. The panel then holds policy only:

- Service profile, which generates the UDP port, ICMP type, GRE peer and IP protocol allowlists.
- Sensitivity, which maps to the baseline multiplier `baseline_k`.
- Block durations and connection caps.
- Geo policy; whitelist and blacklist stay as today.
- Destination and own-prefix overrides. `dst_prefixes` (which destination addresses get their own counters, baselines and Incident state) defaults to the node's addresses on `dev`; `own_prefixes` (your address space, used to drop outside packets claiming a source inside it) defaults to the prefixes you announce. The whitelist is checked before the own-prefix drop, so other Dorian edges and health checkers inside your space still pass.
- Transit capacity and upstream escalation switches (admin only).

Learned baselines and current attack state are shown read-only. The panel gets simpler, and node owners have far fewer ways to weaken protection.